



Getting Started with Tcpdump

This article is an introduction to a popular UNIX tool called *tcpdump*—a very capable command-line utility that allows you to capture network data.

Tcpdump is based on *libpcap*, an open source C/C++ library for network traffic capture. *libpcap* allows you to write portable code by providing a common high-level API for network packet capturing, because almost every operating system offers its own semantics on how to approach low-level network capture functionality.

Running *tcpdump*

You usually have to run *tcpdump* with root privileges—with the help of the *sudo* command—as network capturing is not allowed to everyone, for security reasons. If you try to capture network data using *tcpdump* without having root privileges, you will get the following error message: *tcpdump: no suitable device found*. If you use the *sudo* command, then you will have no problem. You can view the man page with the command *man tcpdump*.

The first thing to do is find out which network interfaces are available on your Linux system; run *tcpdump* with the *-D* option, as shown below:

```
$ sudo tcpdump -D
1.eth0
2.wlan0
3.nflog (Linux netfilter log (NFLOG) interface)
4.eth1
5.any (Pseudo-device that captures on all interfaces)
6.lo
```

On my Mac system running Mac OS X 10.8.2, the command produces the following output:

```
1.en0
2.fw0
3.en1
4.lo0
```

Tcpdump output format

The following is the output from a single packet that was captured using *tcpdump*:

```
20:21:18.347280 IP v-4-kp07-d2084-56.xyzHost.com.http >
192.168.1.10.54796: Flags [.] seq 50400:51840, ack 1, win 33,
options [nop,nop,TS val 1109618191 ecr 1386382236], length 1440
```

Now, let's understand the different parts of the captured packet:

- **20:21:18.347280:** This is the time the capture took place.

As you can easily understand, this is unique to every packet captured by *tcpdump*.

- **IP:** This indicates the use of the IP protocol.
- **v-4-kp07-d2084-56.xyzHost.com.http:** These are the source address and the used port (HTTP) found in the network packet.
- **>:** This symbol separates the source part from the destination part.
- **192.168.1.10.54706:** This is the destination IP address and the port number (54706) used.
- **Flags [.]:** Various flags.
- **seq 50400:51840:** This is the beginning TCP sequence number (50400) and the ending TCP sequence number. TCP uses sequence numbers to order the received data.
- **ack 1:** This is the ACK flag that acknowledges receiving the data from the sender.
- **win 33:** This is the amount of data that will be sent before requiring an ACK packet back from the server.
- **options [nop,nop,TS val 1109618191 ecr 1386382236]:** These are several other options.
- **length 1440:** This is the length of the network packet.

Tcpdump basics

Tcpdump does not capture the entire network packet, by default. This is because, usually, the interest lies in the header parts of the packet that are normally captured with the default packet length.

Here are some of the most useful parameters of *tcpdump*:

- The *-v* parameter produces slightly more output than the default; *-vv* produces even more verbose output; for truly verbose output, use *-vvv*.
- By default, *tcpdump* shows the captured network data on screen. While this is useful sometimes, usually you save the captured data to process it later—using the *-w* parameter followed by the desired filename.
- To read a data file saved by *tcpdump*, use the *-r* option followed by the filename.
- *Tcpdump* uses DNS name resolution by default. If you want to turn that off, use the *-n* parameter. Use *-nn* to turn off both DNS name resolution and port number resolution.
- To capture a given number of packets, use the *-c* parameter.
- The *-ttt* parameter produces a more readable timestamp output, as you can see in the following example:

```
2011-12-26 20:02:12.465078 IP google-public-dns-a.google.com.domain
> 192.168.1.10.53028: 24315 1/0/0 PTR google-public-dns-a.google.
com. (82)
```